

VYTAL secure patient and referral workflow

A practical Windows and VS Code guide for running, testing, and verifying the consent-first patient registry, linked camera screenings, automatic referrals, audit trail, and Supabase row-level security.

WHAT CHANGED	WHY IT MATTERS
Patient directory with explicit consent	A reading is connected to the correct person before capture begins.
Flagged screening to referral workflow	Follow-up is tracked instead of disappearing after the first screening.
Append-only referral event history	Status transitions remain reviewable and browser clients cannot rewrite them.
PostgreSQL RLS by health-worker account	One signed-in account cannot read another account's patient records.

CURRENT CODE TARGET

Repository: github.com/Ahmad-Ali-Shah/Vital

Branch: `agent/secure-database-billing`

Guide date: 19 August 2026

Important: VYTAL is a screening and decision-support prototype. It is not a certified medical device and must not be presented as a diagnosis system.

Run the app in VS Code first

This route uses the built-in memory-only demo data. It lets you test the new UI before connecting Supabase.

1

Open the correct folder

In VS Code choose File > Open Folder and select C:\Users\A\OneDrive\Documents\GitHub\Vital. Then open Terminal > New Terminal.

2

Confirm the branch

The lower-left branch name in VS Code should show agent/secure-database-billing. Run the command below if it does not.

```
git switch agent/secure-database-billing
git status
```

3

Install and start

Use npm.cmd in PowerShell because your Windows execution policy blocks npm.ps1.

```
npm.cmd install
npm.cmd run dev
```

4

Open the site

Keep the terminal running and open <http://localhost:5173> in Chrome or Edge. Allow camera access when asked.

5

Test the patient page

Select Patients in the navigation. Confirm that three demo patients appear, search works, and the Register patient form opens.

6

Test consent

Try creating a patient without ticking the consent checkbox. The app must reject the save. Tick the checkbox, save again, and confirm the new card appears.

7

Test the linked flow

Press Start linked screening on a consented patient. The Scan page must preselect the same patient. Start acquisition and complete the 15-second scan.

Demo-mode behavior

- Demo changes are stored only in memory and disappear after a page refresh.
- A sample referral is already visible on Dashboard, so the workflow can be tested even if your camera reading is not flagged.
- No real patient information should be used in demo mode.

Connect Supabase in the correct order

Use a test Supabase project and fake patient data. Do not use real health information until the privacy, security, and clinical review is complete.

1

Open the Supabase SQL Editor

In your Supabase project select SQL Editor > New query.

2

Apply the core migration

Open the first file in VS Code, copy its complete contents into the SQL Editor, and press Run.

```
supabase/migrations/202608190001_secure_core.sql
```

3

Apply the patient and referral migration

Create another SQL query. Copy and run the complete second file only after the first migration succeeds.

```
supabase/migrations/202608190002_patient_referrals.sql
```

4

Create the browser environment file

At the repository root create .env.local. Use the Project URL and publishable key from Supabase project settings. Never use the service-role or secret key here.

```
VITE_SUPABASE_URL=https://YOUR-PROJECT.supabase.co  
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_YOUR_KEY
```

5

Restart Vite

Stop the existing server with Ctrl+C, answer Y if PowerShell asks, then start it again.

```
npm.cmd run dev
```

6

Create two test accounts

Use Account in VYTAL to create User A and User B. Confirm email addresses if Supabase email confirmation is enabled.

7

Create fake patient data as User A

Sign in as User A, register a fake patient with consent, and complete a linked scan. Sign out before testing User B.

Expected result

- Signed-out users are sent to the secure sign-in screen on Patients.
- User B must not see User A's patient, screening, referral, or audit records.
- Patient and referral rows persist after refresh only when Supabase is configured and the user is signed in.

What to click and what should happen

Test	Action	Expected result
Patient list	Open Patients.	Cards show patient code, name, age, sex, contact, emergency contact, and consent state.
Search	Search by name, then patient code.	Only matching cards remain; the count updates.
Consent blocked	Submit a new patient without consent.	Save is rejected with a readable error.
Consent recorded	Tick consent and register.	Patient is saved with granted status and a consent timestamp.
Migrated patient	Open a pending-consent card and choose Record consent to screen.	A confirmation appears; after approval the card becomes granted.
Patient link	Choose Start linked screening.	The Scan route opens with that patient preselected.
Scan guard	Clear the patient selection and press Start.	Camera does not start; a patient-selection error appears.
Completed scan	Run a clear 15-second face or fingertip scan.	Readings, uncertainty, guidance, and a report link appear.
Referral created	Complete a linked scan that crosses the review threshold.	One referral is created for that screening; duplicate referrals are prevented.
Referral progress	Open Dashboard and advance a referral.	Status moves one step through Flagged, Referred, Contacted, Appointment, and Completed.
Completed referral	Advance to Completed.	Completed styling appears and no further advance button is shown.
Responsive UI	Test at 390 px, 768 px, and desktop widths.	No clipped navigation, overlapping cards, or unreadable referral stages.

Tip: take screenshots of the Patients page, linked Scan page, and referral tracker. These three screens tell the strongest closed-loop care story to judges.

Prove the database is actually isolated

A green UI badge is not proof of database security. Use the following behavioral tests and automated check.

Automated repository checks

```
npm.cmd run security:check
npm.cmd run build
```

Both commands should finish successfully. A large JavaScript chunk warning is a performance warning, not a build failure.

Security test	Method	Pass condition
Authentication gate	Configure Supabase, sign out, and open /patients.	Patient directory is not rendered; sign-in prompt appears.
RLS isolation	Create data as User A, sign in as User B.	User B sees none of User A's rows.
No browser PHI cache	DevTools > Application > Local Storage.	No patient profiles or screening records are stored there.
Consent enforcement	Attempt a scan without selected/granted patient consent.	Capture is blocked before camera initialization.
Audit immutability	Try client insert/update/delete on referral_events.	Supabase rejects the write; authenticated clients have select only.
Secret boundary	Search production bundle and repository.	No Groq, Qwen, Stripe secret, or Supabase service-role key is present.
Payment boundary	Inspect donation flow.	Card numbers and CVC are entered only on provider-hosted Checkout.

Optional SQL inspection

```
select relname, relrowsecurity, relforcerowsecurity
from pg_class
where relname in ('patients', 'referrals', 'referral_events');

select tablename, policyname, roles, cmd
from pg_policies
where tablename in ('patients', 'referrals', 'referral_events')
order by tablename, policyname;
```

The Supabase SQL Editor normally runs with elevated privileges and can bypass RLS. Therefore, the two-account UI test is still required.

Confirm the records behind the UI

After completing a test scan and advancing its referral, inspect only fake test records in the SQL Editor.

Patient and linked screening

```
select p.patient_code, p.full_name, p.consent_status,
       s.id as screening_id, s.status, s.observed_at
from patients p
left join screenings s on s.patient_id = p.id
order by p.created_at desc, s.observed_at desc;
```

Referral and audit trail

```
select r.id, p.patient_code, r.status, r.priority, r.due_at,
       e.from_status, e.to_status, e.occurred_at
from referrals r
join patients p on p.id = r.patient_id
left join referral_events e on e.referral_id = r.id
order by r.created_at desc, e.occurred_at asc;
```

Expected relationships

- screenings.patient_id references the patient owned by the same authenticated account.
- Each flagged screening has at most one referral because screening_id is unique in referrals.
- Every referral status change adds a new referral_events row.
- Browser clients cannot edit or delete referral_events rows.
- A completed referral has completed_at; all non-completed statuses keep it null.

Do not test RLS from the elevated SQL Editor alone. Use separate authenticated app accounts because elevated SQL sessions intentionally bypass normal client policies.

Common problems on your Windows setup

Problem	Fix
npm.ps1 cannot be loaded	Use npm.cmd install and npm.cmd run dev in PowerShell. Do not change execution policy just for this project.
Camera permission denied	Chrome/Edge address bar > camera icon > Allow. Reload the page. localhost is accepted as a secure development origin.
Relation patients does not exist	The second migration was not applied. Run both SQL files in filename order.
Function set_updated_at does not exist	The core migration did not run first. Apply 202608190001_secure_core.sql before the patient migration.
Patients page asks for sign in	This is expected when Supabase is configured. Create or sign into a test account from Account.
Patient is disabled in scan selector	Consent is pending or withdrawn. Open Patients and record consent only after the patient or guardian agrees.
No new referral after scan	A referral is created only when a linked scan crosses the configured review threshold. Use the existing demo referral to test the workflow deterministically.
Changes disappear after refresh	Supabase is not configured or the session is signed out, so the app is correctly using memory-only preview mode.
Build shows chunk size warning	The build still succeeded. Code splitting is a later performance improvement; it does not block Step 1 testing.
Git says nothing to commit	Make sure you are in the Vital folder and on the branch containing these changes. Run git status -sb.

Reset a local demo test

Refresh the browser. Memory-only demo patients and referrals return to their original fixtures. This reset does not delete Supabase data.

Never run destructive SQL such as drop table or truncate against a shared or production database just to reset a demo.

Final checklist before showing judges

☐ npm.cmd run security:check passes.
☐ npm.cmd run build succeeds.
☐ Both Supabase migrations run successfully in order.
☐ Signed-out patient access is blocked.
☐ User B cannot see User A's records.
☐ A patient cannot be registered without explicit consent.
☐ A linked patient remains selected when Scan opens.
☐ Camera permission and the 15-second scan work on the demo laptop and phone.
☐ The Dashboard referral can advance through all five stages.
☐ referral_events contains the status transition history.
☐ No real patient data is used in the presentation.
☐ The team says screening prototype, not diagnosis or certified medical device.
☐ A screen recording or screenshots are ready if venue internet or camera access fails.

Best 90-second demo flow

1. Open Patients and show consent-first registration. 2. Start a linked screening. 3. Show the uncertainty-aware result. 4. Open Dashboard and advance the referral. 5. Explain that RLS isolates every account and the audit trail is append-only.

Step 2 after this sign-off: build a validation and accuracy dashboard using real paired reference-device measurements so judges can see evidence rather than unsupported performance claims.